COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	**SECTION:**	CPE31s2
DATE PERFORMED:	23/07/2026	**DATE SUBMITTED:**	/07/2026
NAME:	Damien Jake D. Gestiada	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **SSH Key-Based Authentication and GIT Setup**

1. PROCEDURE OUTPUT

What procedures did you perform?

Task 1: Create an SSH Key Pair for User Authentication

1. The simplest way to generate a key pair is to run ssh-keygen without arguments. In this case, it will prompt for the file in which to store keys. First, the tool asked where to save the file. SSH keys for user authentication are usually stored in the users .ssh directory under the home directory. However, in enterprise environments, the location is often different. The default key file name depends on the algorithm, in this case id_rsa when using the default RSA algorithm. It could also be, for example, id_dsa or id_ecdsa.

2. Issue the command ssh-keygen -t rsa -b 4096. The algorithm is selected using the -t option and key size using the -b option.
3. When asked for a passphrase, just press enter. The passphrase is used for encrypting the key, so that it cannot be used even if someone obtains the private key file. The passphrase should be cryptographically strong.

4. Verify that you have created the key by issuing the command ls -la .ssh. The command should show the .ssh directory containing a pair of keys. For example, id_rsa.pub and id_rsa.

```
+----[SHA256]-----+
jakeworkstation@workstation:~$ ls -la .ssh
total 24
drwx------  2 jakeworkstation jakeworkstation 4096 Jul 23 00:34 .
drwxr-x--- 16 jakeworkstation jakeworkstation 4096 Jul 14 01:55 ..
-rw-------  1 jakeworkstation jakeworkstation    0 Jul  7 23:38 authorized_keys
-rw-------  1 jakeworkstation jakeworkstation 3389 Jul 23 00:34 id_rsa
-rw-r--r--  1 jakeworkstation jakeworkstation  753 Jul 23 00:34 id_rsa.pub
-rw-------  1 jakeworkstation jakeworkstation 1956 Jul 14 01:55 known_hosts
-rw-------  1 jakeworkstation jakeworkstation 1120 Jul 14 01:34 known_hosts.old
```

Task 2: Copying the Public Key to the remote servers

1. To use public key authentication, the public key must be copied to a server and installed in an authorized_keys file. This can be conveniently done using the ssh- copy-id tool.

2. Issue the command similar to this: ssh-copy-id -i ~/.ssh/id_rsa user@host

Server_1

```
jakeworkstation@workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa.pub jakeserver@192.168.5
6.102
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/jakeworkstation
/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out
any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted n
ow it is to install the new keys
jakeserver@192.168.56.102's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'jakeserver@192.168.56.102'"
and check to make sure that only the key(s) you wanted were added.
```

Server_2

```
jakeworkstation@workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa.pub jakeserver2@192.168.
56.103
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/jakeworkstation
/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out
any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted n
ow it is to install the new keys
jakeserver2@192.168.56.103's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'jakeserver2@192.168.56.103'"
and check to make sure that only the key(s) you wanted were added.
```

3. Once the public key has been configured on the server, the server will allow any connecting user that has the private key to log in. During the login process, the client proves possession of the private key by digitally signing the key exchange.

4. On the local machine, verify that you can SSH with Server 1 and Server 2. What did you notice? Did the connection ask for a password? If not, why?

```
jakeworkstation@workstation:~$ ssh jakeserver@server1
ssh: connect to host server1 port 22: Connection refused
jakeworkstation@workstation:~$ ssh jakeserver@192.168.56.102
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-35-generic x86_64)

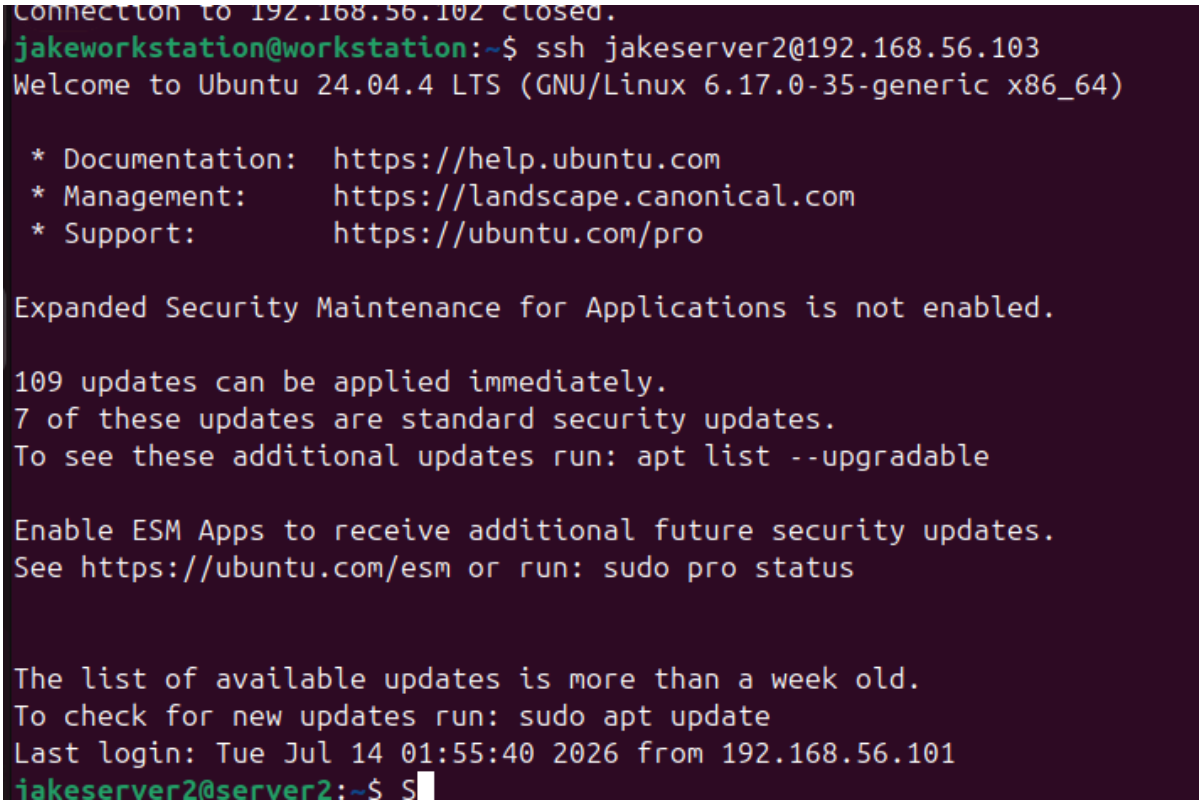
 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

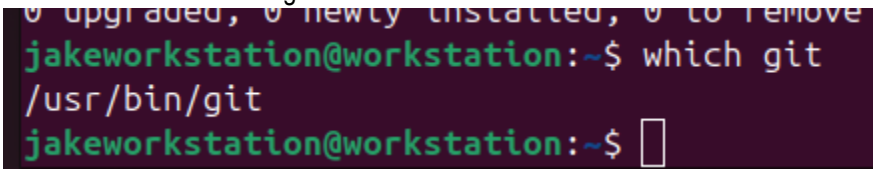

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Last login: Tue Jul 14 02:16:33 2026 from 127.0.0.1
jakeserver@server1:~$ S
```


Connection to 192.168.56.102 closed.
jakeworkstation@workstation:~$ ssh jakeserver2@192.168.56.103
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-35-generic x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro
Expanded Security Maintenance for Applications is not enabled.
109 updates can be applied immediately.
7 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Last login: Tue Jul 14 01:55:40 2026 from 192.168.56.101
jakeserver2@server2:~$ S

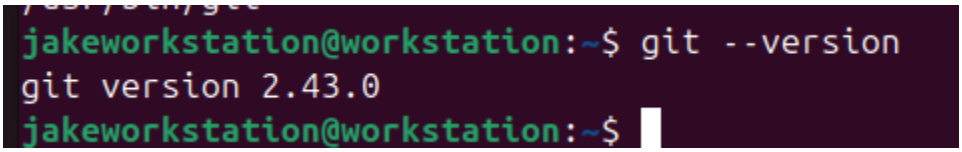
Task 3: Set up the Git Repository

1. On the local machine, verify the version of your git using the command which git. If a directory of git is displayed, then you don't need to install git. Otherwise, to install git, use the following command: sudo apt install git

2. After the installation, issue the command which git again. The directory of git is usually installed in this location: user/bin/git.



3. The version of git installed in your device is the latest. Try issuing the command git --version to know the version installed.



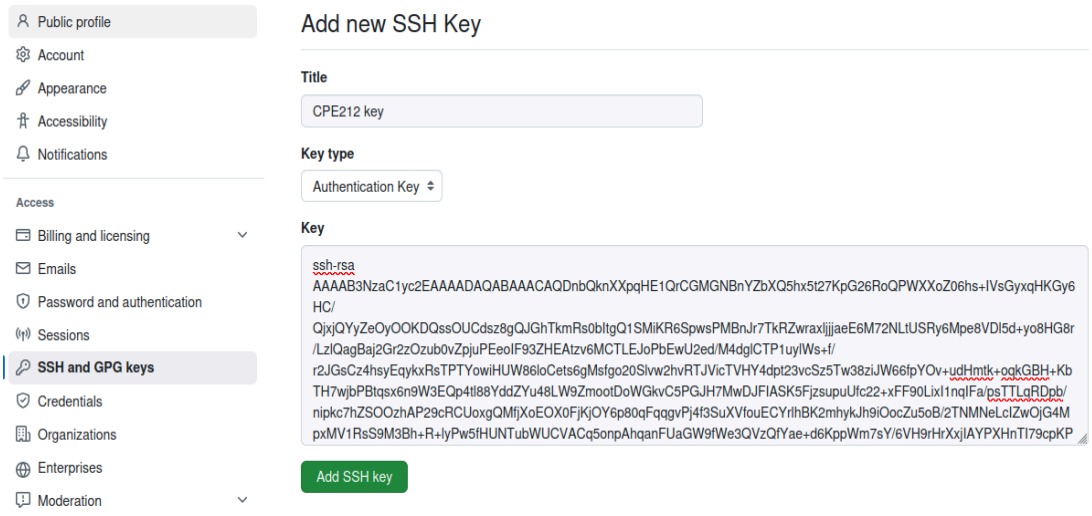
4. Using the browser in the local machine, go to www.github.com.

5. Sign up in case you don't have an account yet. Otherwise, login to your GitHub account.

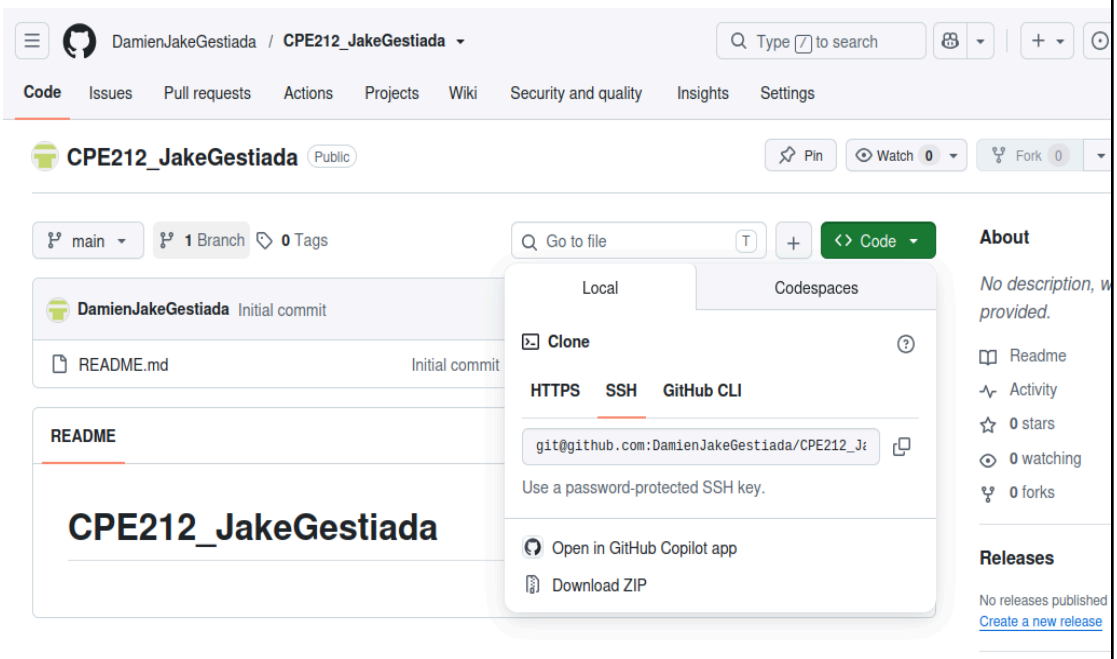
   a. Create a new repository and name it as CPE232_yourname. Check Add a README file and click Create repository.

   b. Create a new SSH key on GitHub. Go your profile's setting and click SSH and GPG keys. If there is an existing key, make sure to delete it. To create a new SSH keys, click New SSH Key. Write CPE232 key as the title of the key.

   c. On the local machine's terminal, issue the command cat .ssh/id_rsa.pub and copy the public key. Paste it on the GitHub key and press Add SSH key.



d. Clone the repository that you created. In doing this, you need to get the link from GitHub. Browse to your repository as shown below. Click on the Code drop down menu. Select SSH and copy the link.



e. Issue the command git clone followed by the copied link. For example, git clone git@github.com:jvtaylar-cpe/CPE232_yourname.git. When prompted to continue connecting, type yes and press enter.

```
jakeworkstation@workstation:~$ git clone git@github.com:DamienJakeGestiada/CPE21
2_JakeGestiada.git
Cloning into 'CPE212_JakeGestiada'...
The authenticity of host 'github.com (20.205.243.166)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

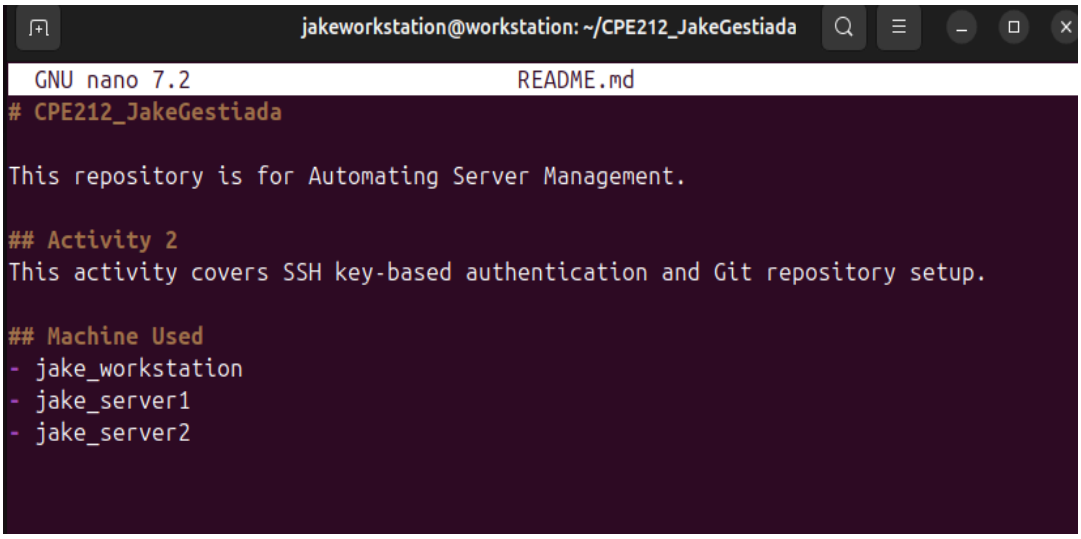
f. To verify that you have cloned the GitHub repository, issue the command ls. Observe that you have the CPE232_yourname in the list of your directories. Use CD command to go to that directory and LS command to see the file README.md.

```
jakeworkstation@workstation:~$ ls
CPE212_JakeGestiada  Documents  Music     Public  Templates
Desktop              Downloads  Pictures  snap    Videos
jakeworkstation@workstation:~$ cd CPE212_JakeGestiada
jakeworkstation@workstation:~/CPE212_JakeGestiada$ ls
README.md
jakeworkstation@workstation:~/CPE212_JakeGestiada$
```

g. Use the following commands to personalize your git.

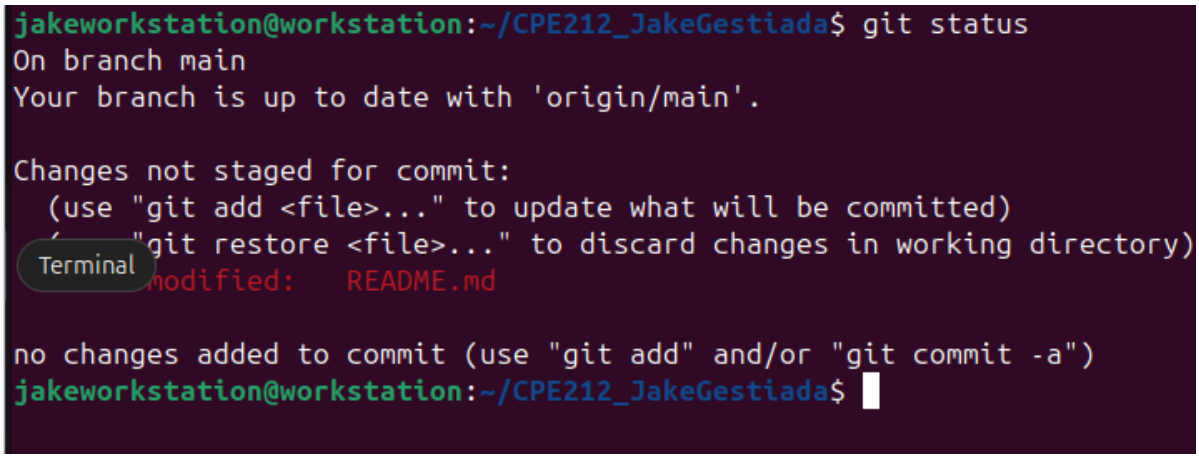
- git config --global user.name “Your Name”
- git config --global user.email yourname@email.com
- Verify that you have personalized the config file using the command cat ~/.gitconfig

```
jakeworkstation@workstation:~/CPE212_JakeGestiada$ git config --global user.name "
Jake Gestiada"
jakeworkstation@workstation:~/CPE212_JakeGestiada$ git config --global user.email
qdjdgestiada@tip.edu.ph
jakeworkstation@workstation:~/CPE212_JakeGestiada$ cat ~/.gitconfig
[user]
        name = Jake Gestiada
        email = qdjdgestiada@tip.edu.ph
jakeworkstation@workstation:~/CPE212_JakeGestiada$
```



h. Edit the README.md file using nano command. Provide any information on the markdown file pertaining to the repository you created. Make sure to write out or save the file and exit.

i. Use the git status command to display the state of the working directory and the staging area. This command shows which changes have been staged, which haven't, and which files aren't being tracked by Git. Status output does not show any information regarding the committed project history. What is the result of issuing this command?



- The result shows that README.md was modified, but it is not yet added to the staging area. It means Git detected changes in the working directory.

j. Use the command git add README.md to add the file into the staging area.

```
jakeworkstation@workstation:~/CPE212_JakeGestiada$ git add README.md
jakeworkstation@workstation:~/CPE212_JakeGestiada$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

jakeworkstation@workstation:~/CPE212_JakeGestiada$
```

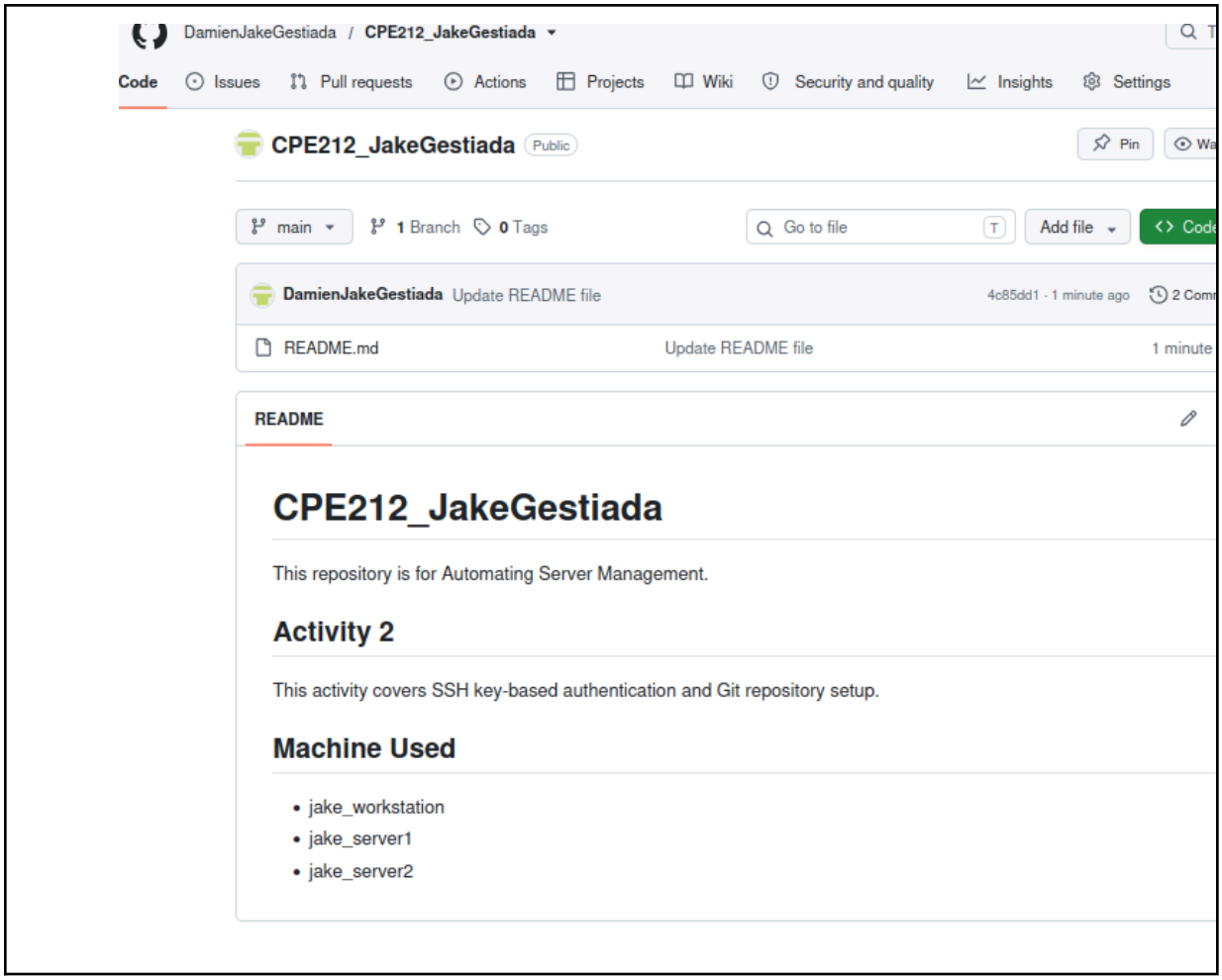
k. Use the git commit -m “your message” to create a snapshot of the staged changes along the timeline of the Git projects history. The use of this command is required to select the changes that will be staged for the next commit.

```
akeworkstation@workstation:~/CPE212_JakeGestiada$ git commit -m "Update README file"
main 4c85dd1] Update README file
1 file changed, 11 insertions(+), 1 deletion(-)
akeworkstation@workstation:~/CPE212_JakeGestiada$
```

l. Use the command git push <remote><branch> to upload the local repository content to GitHub repository. Pushing means to transfer commits from the local repository to the remote repository. As an example, you may issue git push origin main.

```
jakeworkstation@workstation:~/CPE212_JakeGestiada$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 415 bytes | 415.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:DamienJakeGestiada/CPE212_JakeGestiada.git
   1287eb2..4c85dd1  main -> main
```

m. On the GitHub repository, verify that the changes have been made to README.md by refreshing the page. Describe the README.md file. You can notice the how long was the last commit. It should be some minutes ago and the message you typed on the git commit command should be there. Also, the README.md file should have been edited according to the text you wrote.



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

1. How will you describe the ssh-program? What does it do?
   - SSH is an amazing tool that allows us to have safe secure communication through the use of a network with other computers or servers. With the help of my terminal, it gives me access and control to a remote server. Apart from running commands, it facilitates file transfer and server management all done through a terminal. One more advantage is that SSH is not just about accessing the connection but also about safeguarding it through data encryption.
2. How do you know that you already installed the public key to the remote servers?
   - I am sure the public key was installed when I didn't get asked for the user password and was still able to connect to the remote server. A little test would be if I use the ssh command on my local machine and check if a successful connection is established, that means the server accepted my public key. Public key is also saved in the remote server's ~/.ssh/authorized_keys file.
3. What sort of things have we so far done to the remote servers using ansible commands?

- I used Ansible to connect to and test remote servers from my local machine. I was checking if the remote servers can be reached by using Ansible commands. In addition, I can run commands on remote servers without having to manually login to each server, which proved how Ansible can manage several servers much faster and much more easily.

4. How important is the inventory file?
   - The invention file, or better known as an inventory file, is one of the most significant factors because it has the list of remote servers Ansible to work with those remote servers. It could be their server names, IP addresses, usernames, and even how they are grouped. Ansible will look to this file for information about which server is to be used for a command. If there was no inventory file, Ansible would be clueless on which remote machines to connect.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **For command assistance and grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.